

Experiment: Echo Client–Server using TCP/UDP Sockets

1. Aim

To implement an **Echo Client–Server communication using TCP and UDP sockets** and analyze the packet exchange using **Wireshark**.

2. Theory

Socket Programming

A **socket** is an endpoint used for communication between two machines in a network. Socket programming enables applications to send and receive data over a network using protocols like TCP and UDP.

Two major types of sockets are used:

1. TCP (Transmission Control Protocol)

TCP is a **connection-oriented protocol**. Before transmitting data, a connection is established between client and server.

Features:

- Reliable communication
- Error checking
- Data arrives in order
- Uses **three-way handshake**

Working:

1. Client requests connection
2. Server accepts connection

3. Data exchange happens

In an **Echo server**, the server receives a message from the client and sends the **same message back**.

2. UDP (User Datagram Protocol)

UDP is a **connectionless protocol**.

Features:

- No connection setup
- Faster communication
- No guarantee of delivery
- No ordering of packets

UDP is used in:

- Video streaming
 - Online games
 - VoIP applications
-

Echo Client–Server Concept

In an **Echo communication system**:

1. Client sends a message
2. Server receives the message
3. Server sends the **same message back to the client**

This helps in testing network connectivity and understanding socket communication.

3. Requirements

Hardware:

- Computer or Laptop

Software:

- Python 3.x
 - **Wireshark**
 - Code editor (VS Code / PyCharm)
-

4. Procedure

Step 1

Install Python on the system.

Step 2

Install **Wireshark**.

Step 3

Create four Python files:

- tcp_server.py
- tcp_client.py
- udp_server.py
- udp_client.py

Step 4

Start **Wireshark packet capture**.

Step 5

Select network interface (Ethernet / WiFi / Loopback).

Step 6

Run the TCP server program.

Step 7

Run the TCP client program in another terminal.

Step 8

Send messages from client.

Step 9

Observe packets in **Wireshark** using filters:

TCP Filter

tcp.port == 65432

UDP Filter

```
udp.port == 65432
```

Step 10

Stop the capture and analyze packets.

5. Program Code

TCP Server

```
import socket

HOST = '127.0.0.1'
PORT = 65432

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen()

print("Server waiting for connection...")

conn, addr = server.accept()
print("Connected by", addr)

while True:
    data = conn.recv(1024)
    if not data:
        break

    print("Received:", data.decode())
    conn.sendall(data)

conn.close()
```

TCP Client

```
import socket

HOST = '127.0.0.1'
PORT = 65432
```

```
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))
```

```
while True:
    message = input("Enter message: ")
    client.sendall(message.encode())

    data = client.recv(1024)
    print("Echo from server:", data.decode())
```

UDP Server

```
import socket

HOST = '127.0.0.1'
PORT = 65432

server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind((HOST, PORT))

print("UDP Server waiting...")

while True:
    data, addr = server.recvfrom(1024)
    print("Received:", data.decode())
    server.sendto(data, addr)
```

UDP Client

```
import socket

HOST = '127.0.0.1'
PORT = 65432

client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

while True:
    message = input("Enter message: ")
    client.sendto(message.encode(), (HOST, PORT))

    data, server = client.recvfrom(1024)
    print("Echo from server:", data.decode())
```

6. Sample Output

Server Output

Server waiting for connection...

Connected by ('127.0.0.1', 50432)

Received: Hello

Received: Testing

Client Output

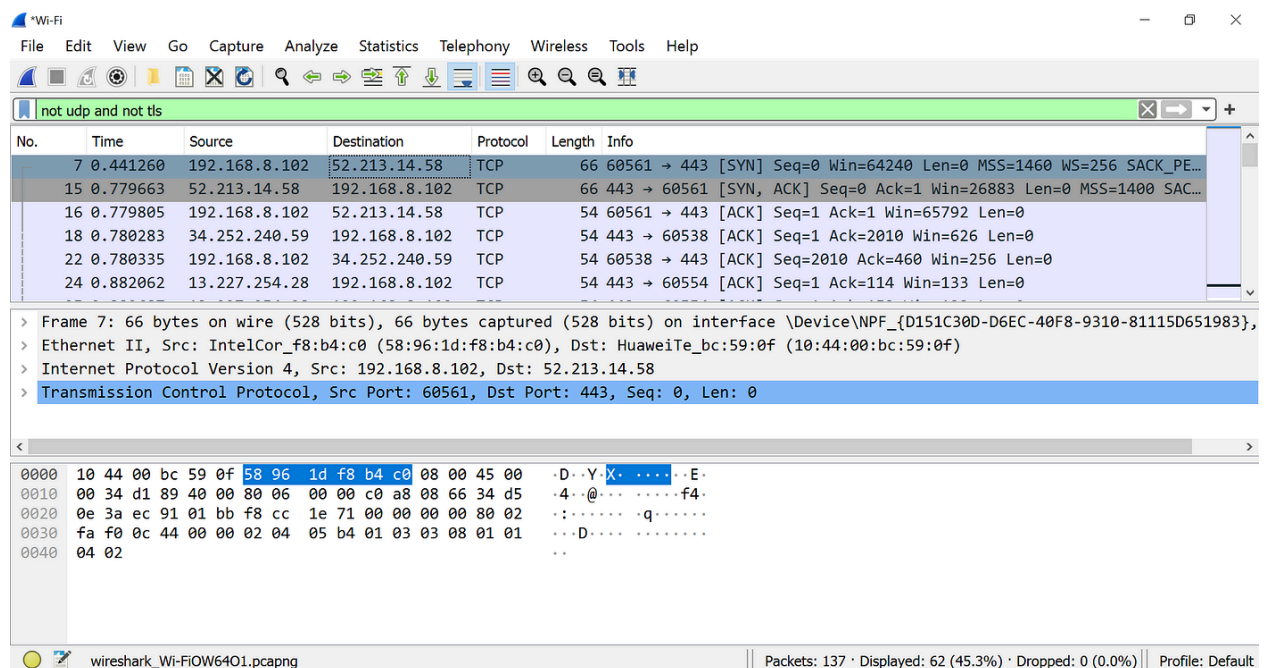
Enter message: Hello

Echo from server: Hello

Enter message: Testing

Echo from server: Testing

7. Wireshark Packet Screenshot Explanation



tv-netflix-problems-2011-07-06.pcap

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/> Expression...

No.	Time	Source	Destination	Protocol	Length	Info
343	65.142415	192.168.0.21	174.129.249.228	TCP	66	40555 → 80 [ACK] Seq=1 Ack=1 Win=5888 Len=0 TSval=491519346 TSecr=551811827
344	65.142715	192.168.0.21	174.129.249.228	HTTP	253	GET /clients/netflix/flash/application.swf?flash_version=flash_lite_2.1&v=1.5&n
345	65.230738	174.129.249.228	192.168.0.21	TCP	66	80 → 40555 [ACK] Seq=1 Ack=188 Win=6864 Len=0 TSval=551811850 TSecr=491519347
346	65.240742	174.129.249.228	192.168.0.21	HTTP	828	HTTP/1.1 302 Moved Temporarily
347	65.241592	192.168.0.21	174.129.249.228	TCP	66	40555 → 80 [ACK] Seq=188 Ack=763 Win=7424 Len=0 TSval=491519446 TSecr=551811852
348	65.242532	192.168.0.21	192.168.0.1	DNS	77	Standard query 0x2188 A cdn-0.nflximg.com
349	65.276870	192.168.0.1	192.168.0.21	DNS	489	Standard query response 0x2188 A cdn-0.nflximg.com CNAME images.netflix.com.edge
350	65.277992	192.168.0.21	63.80.242.48	TCP	74	37063 → 80 [SYN] Seq=0 Win=5840 Len=0 MSS=1460 SACK_PERM=1 TSval=491519482 TSecr=
351	65.297757	63.80.242.48	192.168.0.21	TCP	74	80 → 37063 [SYN, ACK] Seq=0 Ack=1 Win=5792 Len=0 MSS=1460 SACK_PERM=1 TSval=3295
352	65.298396	192.168.0.21	63.80.242.48	TCP	66	37063 → 80 [ACK] Seq=1 Ack=1 Win=5888 Len=0 TSval=491519502 TSecr=3295534130
353	65.298687	192.168.0.21	63.80.242.48	HTTP	153	GET /us/nrd/clients/flash/814540.bun HTTP/1.1
354	65.318730	63.80.242.48	192.168.0.21	TCP	66	80 → 37063 [ACK] Seq=1 Ack=88 Win=5792 Len=0 TSval=3295534151 TSecr=491519503
355	65.321733	63.80.242.48	192.168.0.21	TCP	1514	[TCP segment of a reassembled PDU]

> Frame 349: 489 bytes on wire (3912 bits), 489 bytes captured (3912 bits)

> Ethernet II, Src: Globalsec_00:3b:0a (f0:ad:4e:00:3b:0a), Dst: Vizio_14:8a:e1 (00:19:9d:14:8a:e1)

> Internet Protocol Version 4, Src: 192.168.0.1, Dst: 192.168.0.21

> User Datagram Protocol, Src Port: 53 (53), Dst Port: 34036 (34036)

▼ Domain Name System (response)

Request In: 348

[Time: 0.034338000 seconds]

Transaction ID: 0x2188

> Flags: 0x8180 Standard query response, No error

Questions: 1

Answer RRs: 4

Authority RRs: 9

Additional RRs: 9

▼ Queries

> cdn-0.nflximg.com: type A, class IN

> Answers

> Authoritative nameservers

```

0020  00 15 00 35 84 f4 01 c7 83 3f 21 88 81 80 00 01  ...5....?!.
0030  00 04 00 09 00 09 05 63 64 6e 2d 30 07 6e 66 6c  ....c dn-0.nf
0040  78 69 6d 67 03 63 6f 6d 00 00 01 00 01 c0 0c 00  ximg.com .....
0050  05 00 01 00 00 05 29 00 22 06 69 6d 61 67 65 73  ..... ".images
0060  07 6e 65 74 66 6c 69 78 03 63 6f 6d 09 65 64 67  .netflix .com.edg
0070  65 73 75 69 74 65 03 6e 65 74 00 c0 2f 00 05 00  esuite.n et../...

```

Identification of transaction (dns.id), 2 bytes

Packets: 10299 · Displayed: 10299 (100.0%) · Load time: 0:0.182 · Profile: Default

Wireshark · Follow TCP Stream (tcp.stream eq 0) · test.cap

```

SUBSCRIBE /upnp/service/Layer3Forwarding HTTP/1.1
NT: upnp:event
Callback: <http://192.168.0.2:5000/notify>
Timeout: Second-1800
User-Agent: Mozilla/4.0 (compatible; UPnP/1.0; Windows NT/5.1)
Host: 192.168.0.1
Content-Length: 0
Pragma: no-cache

HTTP/1.0 200 OK
Connection: close
Server: UPnP/1.0 UPnP-Device-Host/1.0
Timeout: Second-1800
SID: uuid:cf

```

3 client pkts, 4 server pkts, 3 turns.

Entire conversation (368 bytes) Show and save data as ASCII Stream 0

Find: Find Next

Help Filter Out This Stream Print Save as... Back Close

When analyzing packets in **Wireshark**, the following steps are observed:

1. TCP Three-Way Handshake

The connection establishment includes:

① SYN Packet

Client sends SYN to request connection.

② SYN-ACK Packet

Server acknowledges and sends SYN.

③ ACK Packet

Client acknowledges the server.

This completes the **TCP connection setup**.

2. Data Transmission

After connection establishment:

- Client sends message packet
- Server receives packet
- Server sends echo response packet

In Wireshark you can see:

Columns:

- Source IP
 - Destination IP
 - Protocol (TCP/UDP)
 - Length
 - Info
-

3. Packet Details Section

Selecting a packet shows:

- Frame information
- Ethernet header
- IP header

- TCP header
 - Data payload
-

4. TCP Stream Analysis

Right click packet → **Follow** → **TCP Stream**

This shows complete communication between client and server.

8. Result

The **Echo Client–Server communication using TCP and UDP sockets** was successfully implemented. The transmitted packets were captured and analyzed using **Wireshark**, confirming successful message exchange between client and server.